Give code for common report where api response can be anything set key as column name and value as data

1 message

Pranav Madhu <pranavmadhu8055@gmail.com> Tue, 3 Nov 2026 at 9:35 am
Draft

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_theme.dart';
import '/flutter_flow/flutter_flow_util.dart';
import '/custom_code/widgets/index.dart'; // Imports other custom widgets
import 'package:flutter/material.dart';
// Begin custom widget code
// DO NOT REMOVE OR MODIFY THE CODE ABOVE!

import 'dart:convert';
import 'package:http/http.dart' as http;

class DynamicCommonReport extends StatefulWidget {
  const DynamicCommonReport({
    super.key,
    this.width,
    this.height,
    required this.apiUrl,
    required this.reportTitle,
  });

  final double? width;
  final double? height;
  final String apiUrl;
  final String reportTitle;

  @override
  State<DynamicCommonReport> createState() => _DynamicCommonReportState();
}

class _DynamicCommonReportState extends State<DynamicCommonReport> {
  bool _isLoading = false;
  String? _errorMessage;

  List<String> _columns = [];
  List> _dataRows = [];

  @override
  void initState() {
    super.initState();
    _fetchReportData();
  }

  Future<void> _fetchReportData() async {
    if (widget.apiUrl.isEmpty) {
      setState(() => _errorMessage = 'API URL is required.');
      return;
    }

    setState(() {
      _isLoading = true;
      _errorMessage = null;
      _columns = [];
      _dataRows = [];
    });

    try {
      final response = await http.get(Uri.parse(widget.apiUrl));

      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw Exception('Failed to load data. Status code: ${response.statusCode}');
      }

      final decoded = jsonDecode(response.body);
      List<dynamic> listData = [];

      // Flexibly handle common API response structures
      if (decoded is List) {
```

```
      listData = decoded;
    } else if (decoded is Map<String, dynamic>) {
      if (decoded.containsKey('result') && decoded['result'] is List) {
        listData = decoded['result'];
      } else if (decoded.containsKey('data') && decoded['data'] is List) {
        listData = decoded['data'];
      } else {
        // If the map itself is the only data, wrap it in a list
        listData = [decoded];
      }
    }

    if (listData.isNotEmpty && listData.first is Map) {
      // Extract columns from the keys of the first row
      // You can also iterate through all rows if keys vary, but usually the first row is sufficient
      final firstRow = listData.first as Map<String, dynamic>;
      _columns = firstRow.keys.toList();

      // Cast all items to Map<String, dynamic>
      _dataRows = listData.map((e) => e as Map<String, dynamic>).toList();
    }

    if (!mounted) return;
    setState(() {
      _isLoading = false;
    });
  } catch (e) {
    if (!mounted) return;
    setState(() {
      _isLoading = false;
      _errorMessage = 'Unable to load report data: $e';
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      leading: IconButton(
        tooltip: 'Back',
        icon: const Icon(Icons.arrow_back, color: Color(0xFF1A202C)),
        onPressed: () {
          if (Navigator.canPop(context)) {
            Navigator.pop(context);
          } else {
            context.go('/'); // Fallback routing if you are using GoRouter
          }
        },
      ),
      centerTitle: true,
      title: Text(
        widget.reportTitle.isNotEmpty ? widget.reportTitle : 'Dynamic Report',
        style: const TextStyle(
          fontWeight: FontWeight.w600,
          fontSize: 20,
          color: Color(0xFF1A202C),
        ),
      ),
      backgroundColor: Colors.white,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black12,
      actions: [
        IconButton(
          icon: const Icon(Icons.refresh, color: Color(0xFF1A202C)),
          onPressed: _isLoading ? null : _fetchReportData,
          tooltip: 'Refresh Data',
        ),
      ],
    ),
    body: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.symmetric(
              horizontal: 24.0,
              vertical: 24.0,
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                if (_errorMessage != null)
```

```
                  Padding(
                    padding: const EdgeInsets.only(bottom: 16.0),
                    child: Text(
                      _errorMessage!,
                      style: TextStyle(
                        color: Colors.red.shade700,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
                if (_isLoading)
                  const Padding(
                    padding: EdgeInsets.symmetric(vertical: 80),
                    child: Center(
                      child: CircularProgressIndicator(
                        color: Color(0xFF005FB8),
                      ),
                    ),
                  )
                else if (_dataRows.isEmpty && _errorMessage == null)
                  const _SectionContainer(
                    title: 'Report Data',
                    icon: Icons.table_chart_outlined,
                    child: Padding(
                      padding: EdgeInsets.symmetric(vertical: 40),
                      child: Center(
                        child: Text(
                          'No data found for this report.',
                          style: TextStyle(color: Color(0xFF4A5568)),
                        ),
                      ),
                    ),
                  )
                else if (_dataRows.isNotEmpty)
                  _buildDynamicTableSection(),
                const SizedBox(height: 32),
              ],
            ),
          ),
        ),
      ],
    ),
  );
}

Widget _buildDynamicTableSection() {
  const headStyle = TextStyle(
    fontSize: 13,
    fontWeight: FontWeight.w700,
    color: Color(0xFF2D3748),
  );
  const cellStyle = TextStyle(
    fontSize: 13,
    color: Color(0xFF1A202C),
  );

  // Formatter to make column keys look like proper titles (e.g. "INT_AUD_STATUS" -> "Int Aud Status")
  String formatColumnName(String key) {
    return key
        .replaceAll('_', ' ')
        .split(' ')
        .map((word) => word.isNotEmpty
            ? '${word[0].toUpperCase()}${word.substring(1).toLowerCase()}'
            : '')
        .join(' ');
  }

  return _SectionContainer(
    title: 'Report Details (${_dataRows.length} Records)',
    icon: Icons.analytics_outlined,
    padding: EdgeInsets.zero, // Remove padding to let the table fill the container
    child: LayoutBuilder(
      builder: (context, constraints) => SingleChildScrollView(
        scrollDirection: Axis.horizontal,
        child: ConstrainedBox(
          constraints: BoxConstraints(minWidth: constraints.maxWidth),
          child: DataTable(
            headingRowColor: WidgetStateProperty.all(const Color(0xFFF9FAFB)),
            dataRowMinHeight: 40,
            dataRowMaxHeight: 72,
            columnSpacing: 24,
            border: const TableBorder(
              horizontalInside: BorderSide(color: Color(0xFFEDF2F7)),
            ),
```

```
              columns: [
                const DataColumn(label: Text('Sr.', style: headStyle)),
                for (final col in _columns)
                  DataColumn(label: Text(formatColumnName(col), style: headStyle)),
              ],
              rows: [
                for (var i = 0; i < _dataRows.length; i++)
                  DataRow(
                    cells: [
                      // Serial Number Column
                      DataCell(Text('${i + 1}', style: cellStyle)),

                      // Dynamic Data Columns
                      for (final col in _columns)
                        DataCell(
                          ConstrainedBox(
                            constraints: const BoxConstraints(
                              minWidth: 80,
                              maxWidth: 300, // Prevent cells from expanding infinitely
                            ),
                            child: Text(
                              _dataRows[i][col]?.toString() ?? '-',
                              style: cellStyle,
                              maxLines: 3,
                              overflow: TextOverflow.ellipsis,
                            ),
                          ),
                        ),
                    ],
                  ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}

// --- Reusable UI Components
class _SectionContainer extends StatelessWidget {
  final String title;
  final IconData icon;
  final Widget child;
  final EdgeInsetsGeometry padding;

  const _SectionContainer({
    required this.title,
    required this.icon,
    required this.child,
    this.padding = const EdgeInsets.all(24.0),
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.05),
            blurRadius: 20,
            offset: const Offset(0, 5),
          ),
        ],
        border: Border.all(color: const Color(0xFFEDF2F7)),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          Container(
            padding: const EdgeInsets.symmetric(
              horizontal: 24.0,
              vertical: 16.0,
            ),
            decoration: const BoxDecoration(
              color: Color(0xFFF9FAFB),
              borderRadius: BorderRadius.only(
                topLeft: Radius.circular(12),
                topRight: Radius.circular(12),
              ),
              border: Border(
                bottom: BorderSide(color: Color(0xFFEDF2F7)),
```

```
            ),
          ),
          child: Row(
            children: [
              Icon(
                icon,
                size: 22,
                color: const Color(0xFF4A5568),
              ),
              const SizedBox(width: 12),
              Expanded(
                child: Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                    color: Color(0xFF2D3748),
                  ),
                  overflow: TextOverflow.ellipsis,
                ),
              ),
            ],
          ),
        ),
        Padding(
          padding: padding,
          child: child,
        ),
      ],
    ),
  );
 }
}
```